Advertised Learning Outcomes:

Module 1

- understand how variables are stored and manipulated by your program;
- know the limitations of Python's built-in list structure and how it compares to other languages' array structures;
- know how to use lists/arrays effectively in circular and multidimensional contexts
- be able to sort values in two different ways and understand each approach's performance characteristics;
- know how to use common notation to describe performance;
- derive a custom structure to overcome some of the limitations of lists/arrays.

Module 2

- understand the benefits and drawbacks of restricting access to items within a data structure;
- know when attributes and methods should be public or private;
- know how to apply recursion (functions calling themselves) for basic problem solving;
- know when to apply last-in-first-out and first-in-first-out ordering restrictions for data sequences;
- be able to make business management decisions based on statistical analysis of certain data structures;
- be familiar with exponential-time performance and its associated detriments;
- apply several new software engineering patterns to improve the flexibility and robustness of your programs; and
- dramatically improve the quality of your test code, and by extension, your programs themselves.

Module 3

- understand hierarchical arrangements of data using familial terms such as parent, child, and sibling;
- store arithmetic expressions in a format that obviates order of operations;
- solve arithmetic expressions in a particular format;
- visualize the worst case inputs for tree structures and know how to adapt to them;
- be able to maintain a sorted order of data implicitly via particular structures;
- sort an arbitrary sequence of data in optimal time;
- encode an arbitrary string of characters using the smallest possible amount of space.

Module 4

- be able to locate arbitrary data in constant time on average (worst case is still at least logarithmic),
- understand how dictionaries work and be able to construct them as custom types,
- construct a search tree of arbitrary order as opposed to binary only, and
- understand at a high level how file directories work in an operating system.

ACM CS2 Curriculum 2023 Learning Outcomes
Items covered by CSCI 241 highlighted in blue
CS Core: (See also: SDF-Data-Structures, SDF-Algorithms)

1. Abstract Data Type (ADT) and operations on an ADT (See also: FPL-Types)
    a. Dictionary operations (insert, delete, find)
2. Arrays
    a. Numeric vs non-numeric, character strings
    b. Single (vector) vs multidimensional (matrix)
3. Records/Structs/Tuples and Objects (See also: FPL-OOP)
4. Linked lists (for historical reasons)
    a. Single vs Double and Linear vs Circular
5. Stacks
6. Queues and deques
    a. Heap-based priority queue
7. Hash tables/maps
    a. Collision resolution and complexity (e.g., probing, chaining, rehash)
8. Graphs (e.g., [un]directed, [a]cyclic, [un]connected, and [un]weighted) (See also: MSF-Discrete)
    a. Graph representation: adjacency list vs matrix
9. Trees (See also: MSF-Discrete)
    a. Binary, n-ary, and search trees
    b. Balanced (e.g., AVL, Red-Black, Heap)
10. Sets (See also: MSF-Discrete)
11. Search algorithms
    a. $O(n)$ complexity (e.g., linear/sequential array/list search)
    b. $O(log_2 n)$ complexity (e.g., binary search)
    c. $O(log_b n)$ complexity (e.g., uninformed depth/breadth-first tree search)
12. Sorting algorithms (e.g., stable, unstable)
    a. $O(n^2)$ complexity (e.g., insertion, selection),
    b. $O(nlogn)$ complexity (e.g., quicksort, merge, timsort)
13. Graph algorithms
    a. Shortest path (e.g., Dijkstra's, Floyd's)
    b. Minimal spanning tree (e.g., Prim's, Kruskal's)

KA Core:

14. Sorting algorithms
    a. $O(nlogn)$ complexity heapsort
    b. Pseudo $O(n)$ complexity (e.g., bucket, counting, radix)
15. Graph algorithms
    a. Transitive closure (e.g., Warshall's)
    b. Topological sort
16. Matching
    a. Efficient string matching (e.g., Boyer-Moore, Knuth-Morris-Pratt)
    b. Longest common subsequence matching
    c. Regular expression matching

Non-core:

17. Cryptography algorithms (e.g., SHA-256) (See also: SEC-Crypto)

18. Parallel algorithms (See also: PDC-Algorithms, FPL-Parallel)
19. Consensus algorithms (e.g., Blockchain) (See also: SEC-Crypto)
    a. Proof of work vs proof of stake (See also: SEP-Sustainability)
20. Quantum computing algorithms (See also: AL-Models, AR-Quantum)
    a. Oracle-based (e.g., Deutsch-Jozsa, Bernstein-Vazirani, Simon)
    b. Superpolynomial speed-up via QFT (e.g., Shor's)
    c. Polynomial speed-up via amplitude amplification (e.g., Grover's)
21. Fast-Fourier Transform (FFT) algorithm
22. Differential evolution algorithm

Illustrative Learning Outcomes:

CS Core:

1. For each ADT/Data-Structure in this unit
    a. Explain its definition, properties, representation(s), and associated ADT operations.
    b. Explain step-by-step how the ADT operations associated with the data structure transform it.
2. For each algorithm in this unit explain step-by-step how the algorithm operates.
3. For each algorithmic approach (e.g., sorting) in this unit apply a prototypical example of the approach (e.g., merge sort).
4. Given requirements for a problem, develop multiple solutions using various data structures and algorithms. Subsequently, evaluate the suitability, strengths, and weaknesses selecting an approach that best satisfies the requirements.
5. Explain how collision avoidance and collision resolution is handled in hash tables.
6. Explain factors beyond computational efficiency that influence the choice of algorithms, such as programming time, maintainability, and the use of application-specific patterns in the input data.
7. Explain the heap property and the use of heaps as an implementation of a priority queue.

KA Core:

8. For each of the algorithms and algorithmic approaches in the KA Core topics.
    a. Explain a prototypical example of the algorithm, and
    b. Explain step-by-step how the algorithm operates.

Non-core:

9. An appreciation of quantum computation and its application to certain problems.

CSCI 241 Data Structures
Fall 2026 Syllabus

Course Staff and Meeting Information

Kevin Coogan, instructor
ISC 2335

Jim Deverick, instructor
ISC 2367

Brent Thal, instructor
ISC 2380

Ling Ding, TA
Yongkang Wang, TA
Siyuan Zhao, TA

section 01: TR 1100—1220 Boswell 341
section 02: TR 1230—1350 ISC 1360
section 03: TR 1400—1520 ISC 1360
section 04: TR 1700—1820 ISC 1360

instructor office hours
M 1300—1430 Deverick ISC 2367
R 0900—1030 Deverick ISC 2367

MW 1000—1100 Coogan ISC 2335
TR 1030—1130 Coogan ISC 2335

TR 1230—1650 Thal ISC 2380

The best means of reaching us is through private Piazza posts. We ask that you use this medium primarily for communications.

Required Materials

- Goodrich, Tamassia, and Goldwasser. Data Structures & Algorithms in Python. Wiley, 2013. ISBN 978-1-118-29027-9. This textbook is available online at no cost to you through Swem Library. We do not use any digital content from the textbook that requires paid access.
- Course materials will be distributed through Blackboard.
- Most items will be submitted on paper or electronically through Gradescope. You already have an account with Gradescope through your W&M login credentials. All items will be returned via Gradescope.
- We will use Piazza for class discussion and questions. You will receive an e-mail invitation to Piazza; please accept it. Piazza has changed their model to one that asks for voluntary contributions. This is a function of Piazza and is not under the control of the instructors. You are not asked by the instructors nor are you required by this course to contribute to Piazza.
- Programming in this course will be done in Python3 using miniforge3.
- Programming in this course is done using a text editor and the command line. We suggest VSCode, which is available for free. For command line environments, Mac users will employ Terminal; Windows users will use PowerShell. Both are installed by default.

Exam Information

See the course schedule at the end of this document for exam section details. Switching exam sections requires approval from the office of Academic Wellbeing.

Readings, Practice Problems, and Quizzes

Regular reading assignments from the textbook and academic papers appear on the course calendar at the end of this document alongside lecture content. There is material in the assigned readings that will appear on quizzes and the exam, regardless of whether that material is covered in class.

We will issue practice problems for completion outside of class to reinforce the material we cover or assign. Although these items do not impact your grade, we may present them in electronic form that provides correctness feedback.

There will be several short paper quizzes. The initial versions will be completed in class. If you are not satisfied with your performance, you may retake quizzes (with different problems) in several evening sessions that will be available throughout the semester. Quizzes can only be completed during the in-class or retake sessions. We do not permit individual retakes or make-up sessions.

Programming Projects and Slip Days

In addition to implementing the abstract data types that we discuss, you will write small applications that employ those data types to see how they are used. Most programming projects will take between two and three weeks to complete and they increase in complexity as the course moves forward. The submission environment will provide automated functionality tests; you may submit as many times as you like until the deadline to improve your functional score. Other aspects of your programming projects, such as performance, readability, and adherence to specifications will be evaluated manually. You are required to use Python 3.x for this course.

Each person will receive an allocation of two slip days to adjust project deadlines without penalty. Each applied slip day extends the deadline for a project by exactly 24 hours and cannot be divided into smaller extensions. Slip days are automatically applied to late submissions. Unused slip days have no value. If you submit a project late and have no slip days remaining, you will be penalized 25% of the project's total value per 24-hour period or part thereof beginning one minute past the posted deadline.

Some programming project submissions also include brief prose sections meant to illustrate both your theoretical and your practical understanding of the structures we create and apply, and the techniques used to implement them including the use of AI. We will provide questions or other prompts to stimulate your responses, which will constitute a significant portion of your project scores.

We allow you to submit an unlimited number of times, but we only evaluate your latest submission.

Artificial Intelligence

We recognize that artificial intelligence has changed the landscape of computer programming. Part of becoming a successful developer is learning how to use these tools effectively while understanding how the output that they generate works and confirming that it is correct. For the programming projects in this course, you may employ AI tools such as ChatGPT, Claude, or others to help you generate code that conforms to the specifications, but you must be able to demonstrate that you understand how and why that code works. To achieve the strongest learning outcomes, we recommend working with AI in small, interactive iterations that focus on components of the specifications rather than prompting it with the entire specification at once. For each project, we ask that you submit a brief explanation of how you employed AI including a discussion of your most effective interaction with it. Pay close attention to the specifications for each project for guidelines on how to use AI in that project most effectively.

We also recommend that you work with AI to help visualize structures during insertion and removal of data. In this application, it is imperative to specify the parameters of our approach carefully and to critique the generated output for correctness. Recognize that like any source, AI may sometimes be incorrect or may make alternate assumptions about the rules that govern certain data structures. While AI can be a helpful companion tool, it does not replace your own analysis and independent thought.

AI-assisted Term Project

As we learn various data structures throughout the semester, you'll be given options for a term project that is more complex than our regular programming projects. This may involve implementing a more advanced data structure that is a bit different from what we cover in class, or it may be an application that employs the content we cover. For this project, you may use your AI engine of choice as much as you like to generate and test code. Be sure to save your exchanges with AI (both prompts and responses), as these will be submitted with your implementation. Finally, you will submit a short video explanation of how your project works, how AI assisted or hindered your progress, and how our concepts from class are applied in the project.

Attendance, Accessibility, and Honor

You are expected to be present and in good hygienic condition at every class meeting. We may take roll, including electronically. Please have your ID with you at all class meetings. Missing more than two meetings is cause for concern and may result in a grade penalty.

It is the policy of William & Mary to accommodate students with disabilities and qualifying diagnosed conditions in accordance with federal and state laws. Any student who feels s/he may need an accommodation based on the impact of a learning, psychiatric, physical, or chronic health diagnosis should contact us privately to discuss your specific needs. Students will also need to contact Student Accessibility Services staff at 757-221-2512 or at sas@wm.edu to determine if accommodations are warranted and to obtain an official letter of accommodation. For more information, please see www.wm.edu/sas. You have our word that such matters will be handled professionally and with sensitivity. We associate no stigma with any form of disability.

Please keep in mind that you are bound by all aspects of the College's Honor Code, to which you have pledged your support. We will bring instances of academic dishonesty to the attention of the Honor Council.

Generally, we maintain an empty-hands policy for collaboration. You are free to discuss concepts with your colleagues *currently enrolled* in CSCI 241 at W&M, but you must not have any materials in front of you when you do. **No person other than you, the instructors, the TAs, or the lab instructors or consultants should ever see your work.** This means that relatives, students at other schools, alumni of the course (other than consultants), and online platforms such as CourseHero, Chegg, and others are prohibited. If you discuss something with permitted individuals using a whiteboard or paper or anything electronic, you must erase or destroy any collaboratively created results immediately when finished and without recording it in any other form, including photography. The granularity of collaborative discussions must be coarse enough that two people walking away from a conversation can reasonably be expected to produce different code. **If you see another student's work or allow your work to be seen by another student, then we will pursue Honor Council charges against both of you.**

We employ automated tools to check for plagiarism in both programming and written work.

Grading Policies

Quizzes are scored and returned in Gradescope. If you believe we have assigned a score incorrectly, you may request a regrade from within Gradescope. Be honorable in your requests; do not ask for credit that you did not earn.

Projects are evaluated using automated functionality tests and manual human evaluation with written (electronic) feedback, also in Gradescope. We employ blind grading; the graders do not know whose work they are evaluating. To enable blind grading, it is important that you not include any personally identifying marks anywhere in your submission except where specified.

Your grade in this course will be computed as follows:

Quizzes	42%
Programming Projects	28%
Final Exam	20%
AI-assisted Term Project	10%

Letter grades are issued based on the following rubric:

A	At least 94%	C	At least 74% but less than 77%
A-	At least 90% but less than 94%	C-	At least 70% but less than 74%
B+	At least 87% but less than 90%	D+	At least 67% but less than 70%
B	At least 84% but less than 87%	D	At least 64% but less than 67%
B-	At least 80% but less than 84%	D-	At least 60% but less than 64%
C+	At least 77% but less than 80%	F	Less than 60%

Letter grades are available in Blackboard throughout the course and are updated as items are returned.

Mental and Physical Well-Being

William & Mary recognizes that students juggle different responsibilities and can face challenges that make learning difficult. There are many resources available at W&M to help students navigate emotional/psychological, physical/medical, material/accessibility concerns, including:

- The W&M Counseling Center at (757) 221-3620. Services are free and confidential.
- The W&M Health Center at (757) 221-4386.
- For additional support or resources & questions, contact the Dean of Students at 757-221-2510.
- For a list of other resources available to students, please see the Office of Health & Wellness webpage.

Covered Topics

The purpose of this course is to introduce you to abstract data representation. You will learn how to represent data for various programming problems, and how to analyze the performance impacts of your data representation decisions. Though algorithms are not the primary focus of this course, we will introduce some well-known examples to illustrate the merits and performance of various data structures. The general tentative schedule for the course follows.

Course Week	Content	Read Before Class	Practice Before Class	Progress	Submit projects before 2359 Quizzes in class
0	Introductions and Course Overview				
1	1.1 Python Variables and Arrays and Project 1 Overview	Ch. 1			
1	1.2 Circular Arrays	5.1, 5.4, 5.5	Ckpt. 1.1	P1 Insertion sort complete	
2	1.3 Multidimensional Arrays	5.6	Ckpt. 1.2	P1 Selection sort complete	
2	1.4 Performance Analysis	3.1, 3.2, 3.3	Ckpt. 1.3	P1 Writeup begun	
Sept. 13	**PROJECT DEADLINE**				**Project 1**
3	1.5 Linked Lists	7.1, 7.2, 7.3	Ckpt. 1.4		**Arrays Quiz**
3	1.6 Linked Lists				
4	Project 2 Iteration, Exceptions, and Testing Overview			P2 append_element, __str__ complete with testing	

Course Week	Content	Read Before Class	Practice Before Class	Progress	Submit projects before 2359 Quizzes in class
4	2.1 Stacks	6.1, 7.1.1	Ckpt. 1.6	P2 insert_element_at, get_element_at, remove_element_at complete with testing	
5	2.3 Recursion	Ch. 4	Ckpt. 2.1	P2 __iter__, __next__ complete with testing; P2 Applications complete; all bugs fixed	**Linked Lists Quiz**
5	2.2 Queues and Deques	6.2, 6.3, 7.1.2, 7.2.2, 7.3.2	Ckpt. 2.3		
Oct. 4	**PROJECT DEADLINE**				**Project 2**
6	Project 3 Overview		Ckpt. 2.2	P3 Array_Deque including __grow complete with test cases.	**Stacks Quiz**
6	2.4 Queue Statistics			P3 Linked_List errors fixed, Linked_List_Deque complete with test cases.	
7	3.1 Binary Trees	Ch. 8	Ckpt. 2.4	P3 Stack, Queue complete with test cases.	
7	3.2 Binary Search Trees	Ch. 8, 11.1	Ckpt. 3.1		
8	Project 4 Overview		Ckpt. 3.2	P3 Applications complete.	**Recursion Quiz**

Course Week	Content	Read Before Class	Practice Before Class	Progress	Submit projects before 2359 Quizzes in class
8	3.3 Heaps	9.1, 9.2, 9.3, 9.4			
Oct. 25	**PROJECT DEADLINE**				**Project 3**
9	3.4 Huffman Trees	Academic Paper	Ckpt. 3.3	P4 insert_element and in_order complete with in_order test cases	**Expression Trees and Binary Search Trees Quiz**
9	3.5 Balanced Binary Search Trees	11.3, Academic Paper	Ckpt. 3.4	P4 pre_order, post_order complete with test cases	
10	3.5 Balanced Binary Search Trees		Ckpt. 3.5	P4 remove_element, get_height complete with get_height test cases	
10	Catch up/Review			P4 remove_element test cases	
Nov. 8	**PROJECT DEADLINE**				**Project 4 Milestone**
11	Catch up/Review			P4 __balance single rotations complete with test cases	
11	4.1 Hash Tables	10.2		P4 __balance double rotations complete with test cases	
12	4.2 B+Trees	15.3	Ckpt. 4.1	P4 to_list, applications complete	**Balanced Binary Search Trees Quiz**
12	Catch up/Review		Ckpt. 4.2		

Course Week	Content	Read Before Class	Practice Before Class	Progress	Submit projects before 2359 Quizzes in class
Nov. 19	**PROJECT DEADLINE**				**Project 4 Final**
13	Data Structures in the Wild				
13	Final Exam Review Session				
Dec. 4	**PROJECT DEADLINE**				**Term Project**
Dec 7	**Section 01 FINAL EXAM 1400—1700**				
Dec 9	**Section 02 FINAL EXAM 0900—1200**				
Dec 10	**Section 03 FINAL EXAM 1400—1700**				
Dec 10	**Section 04 FINAL EXAM 1900—2200**				